



**UNIVERSIDAD
DE GRANADA**

Práctica 6

SBC alimentación

Pablo Linari Pérez

DNI: 75571079W

Curso: 25/26

Correo: pablolinari@correo.ugr.es

Índice de contenidos

1	Datos del trabajo	3
2	Resumen de cómo funciona el sistema	3
3	Descripción del proceso seguido	4
3.1	Procedimiento seguido para el desarrollo de la base de conocimiento	4
3.2	Procedimiento de validación y verificación del sistema	4
4	Descripción del sistema	5
4.1	Variables de entrada del problema y su representación	5
4.2	Variables de salida del problema y su representación	6
4.3	Conocimiento global del sistema (carga inicial)	7
4.4	Especificación de los módulos	7
4.4.1	Estructura en módulos	7
4.4.2	Módulo 1 — Deducción de propiedades de las recetas	7
4.4.3	Módulo 2 — Adquisición de información del usuario	8
4.4.4	Módulo 3 — Filtrado por restricciones esenciales	8
4.4.5	Módulo 4 — Valoración por factores de certeza	8
4.4.6	Módulo 5 — Propuesta de receta y reajustes	9
5	Manual de uso del sistema	9

1 Datos del trabajo

El sistema está implementado en CLIPS y se reparte en dos ficheros:

- `SBC_recetas.clp`: base de conocimiento, motor de inferencia y diálogo con el usuario.
- `BDrecetas_100.clp`: base de hechos con 100 recetas reales (hechos `receta` e `ingrediente`) extraídos cada uno de una página web con su correspondiente enlace.

Las instrucciones de ejecución del programa se dan al final de la memoria.

2 Resumen de cómo funciona el sistema

El sistema es un asistente experto interactivo que recomienda una receta de un recetario de unas 100 recetas a partir de las preferencias y restricciones del usuario.

Su funcionamiento se divide en dos fases:

1. **Preproceso de la base de conocimiento.** Antes de hablar con el usuario, el sistema analiza cada receta y deduce automáticamente propiedades que no vienen dadas explícitamente: si es vegetariana/vegana, si contiene gluten o lactosa, si es picante, su nivel de calorías y de proteínas, su ingrediente relevante, e incluso el tipo de plato cuando no está informado. Estas deducciones se hacen con reglas de alta prioridad (`salience 2000/1900`) para que estén disponibles cuando comience el diálogo.
2. **Diálogo y razonamiento con incertidumbre.** El sistema pregunta al usuario, justificando por qué hace cada pregunta. Distingue dos clases de información:
 - **Restricciones** (dieta, sin lactosa, sin gluten, ingrediente obligatorio): conocimiento cierto. Una receta que las incumple se descarta.
 - **Preferencias** (tipo de plato, proteína, densidad calórica, comensales, tiempo): se tratan como conocimiento incierto. No descartan recetas, sino que aportan evidencia a favor o en contra mediante un factor de certeza (CF) en $[-1, 1]$ de esta manera al final conseguimos asignar distintos CF a cada receta y se escoge el que mejor resultado tenga.

Los factores de certeza de cada receta se combinan con la fórmula de MYCIN y se recomienda la receta de mayor CF combinado, explicando al usuario cada aporte de evidencia. El usuario puede aceptar la receta, rechazarla (se ofrece la siguiente mejor) o reajustar un criterio y relanzar el razonamiento.

El sistema integra de forma combinada los cuatro tipos de razonamiento con incertidumbre vistos en la asignatura:

- **Razonamiento por defecto:** si el usuario no da ninguna preferencia, se asume por defecto que prefiere recetas fáciles.
- **Lógica difusa:** el encaje de la densidad calórica se mide con el grado de pertenencia de las kcal reales a conjuntos difusos trapezoidales.
- **Factores de certeza (MYCIN):** motor principal de decisión, se utiliza para calcular el CF combinado .

- **Razonamiento probabilístico:** como segunda opinión, estima la probabilidad de acierto como la media geométrica de la satisfacción por aspecto ,cada CF se traduce a una probabilidad y se promedian geométricamente.

3 Descripción del proceso seguido

3.1 Procedimiento seguido para el desarrollo de la base de conocimiento

1. **Adquisición y representación del recetario.** Se recopilaron unas 100 recetas reales con su enlace web y se representaron como hechos `receta` e `ingrediente` en `BDrecetas_100.clp`. Cada receta guarda únicamente los datos objetivos y fáciles de obtener: nombre, tipo de plato, dificultad, comensales, tiempo de cocinado, información nutricional (texto libre) e ingredientes con su cantidad y unidad.
2. **Identificación del conocimiento derivado.** Se observó que muchas propiedades útiles para recomendar apta para una dieta, contiene alérgenos, nivel calórico/proteico... no conviene introducirlas a mano receta a receta, porque sería costoso y produciría errores. Por eso se decidió deducirlas mediante reglas a partir de los ingredientes y la información nutricional.
3. **Construcción del vocabulario de ingredientes.** Se han definido funciones auxiliares (`es-condimento`, `es-importante`, `es-dulce`, `contiene`) y listas de palabras clave (`carne`s, `pescado`s, `lácteo`s, `harina`s, `picante`s...) que permiten clasificar cada receta inspeccionando los nombres de sus ingredientes mediante búsqueda de subcadenas.
4. **Definición de las reglas de deducción de propiedades:** Primero se buscan marcadores que indiquen si algo está presente , por ejemplo alimentos obligatorios , después se usa otro nivel para deducir las propiedades que se identifican con negación o afirmación , por ejemplo (`es vegetariana`/`vegana`/`sin gluten`/`sin lactosa`).
5. **Diseño del razonamiento con incertidumbre.** Se adaptaron los ejemplos de la asignatura (`ejemplofuzzy.clp`, factores de certeza y razonamiento por defecto) para puntuar las recetas que superan el filtrado según los atributos que se usan para las preferencias.
6. **Refinamiento.** Se ajustaron los umbrales (calorías baja/media/alta, proteína, conjuntos difusos, valores de CF) probando el sistema con distintas combinaciones de respuestas para ver que razonamiento se ajustaba mejor al objetivo del sistema.

3.2 Procedimiento de validación y verificación del sistema

- **Verificación de la carga:** la regla `comprobar-recetas-cargadas` detecta si no se ha cargado el recetario y avisa al usuario, evitando ejecuciones sin datos.
- **Verificación de las deducciones:** tras (`reset`) y (`run`) se inspeccionó la base de hechos (`((facts))`) para comprobar que las propiedades deducidas (`propiedad_receta ...`) eran correctas para una muestra de recetas conocidas (p. ej. que una receta con

jamón quedaba marcada `no_vegetariana`, o que un postre con harina quedaba `contiene_gluten`).

- **Validación de la entrada del usuario:** cada pregunta tiene su regla de **validación** (`validar-dieta`, `validar-tipo`, `validar-comensales`...) que rechaza valores fuera del dominio y repite la pregunta, garantizando robustez ante respuestas erróneas.
- **Validación del filtrado:** se probaron restricciones (dieta vegana, sin gluten, ingrediente obligatorio) verificando que las recetas incompatibles desaparecían de las candidatas.
- **Validación del razonamiento:** se comprobó que la receta propuesta era siempre la de mayor CF entre las compatibles, que la justificación generada coincidía con las respuestas dadas, y que las preguntas que no discriminaban se omitían correctamente (reglas `omitir-****-si-no-aporta`).
- **Validación de casos límite:** sin ninguna preferencia (todo `ns`, se aplica el razonamiento por defecto), respuesta `fin` anticipada, y el caso de no quedar recetas compatibles (se ofrece modificar un parámetro).

A continuación se muestran algunos de los casos de prueba empleados para validar el sistema, combinando distintas restricciones y preferencias:

Dieta	Lactosa	Gluten	Ingrediente	Comensales	Tiempo	Tipo
normal	ns	si	atún	5	40	entrante
vegana	ns	no	ns	2	20	principal
normal	ns	no	ns	3	19	postre
vegetariana	si	ns	cebolla	ns	15	principal
normal	si	si	zanahoria	1	10	entrante
normal	no	si	pollo	2	5	principal
normal	no	si	ns	2	20	ns
ns	ns	ns	ns	ns	ns	ns
ns	si	no	harina	10	30	postre
ns	ns	ns	ns	ns	ns	ns
vegana	ns	fin				
normal	no	si	arroz	fin		

4 Descripción del sistema

4.1 Variables de entrada del problema y su representación

Las entradas son las respuestas del usuario, recogidas como hechos ordenados o plantillas. Todas admiten `ns` (sin preferencia) y `fin` (terminar y recomendar con lo respondido). Se distinguen **restricciones** (filtrado duro) y **preferencias** (factores de certeza):

Variable	Clase	Valores	Representación
Dieta	Restricción	normal / vegetariana / vegana / ns / fin	(respuesta-dieta ?v)
Sin lactosa	Restricción	si / no / ns / fin	(respuesta-sin-lactosa (valor ?v))
Sin gluten	Restricción	si / no / ns / fin	(respuesta-sin-gluten (valor ?v))
Ingrediente obligatorio	Restricción	texto / ns / fin	(respuesta-ingrediente-obligatorio ?v)
Tipo de plato	Preferencia	principal / entrante / postre / ns / fin	(respuesta-tipo ?v)
Comensales	Preferencia	número / ns / fin	(respuesta-comensales ?v)
Tiempo máx. (min)	Preferencia	número / ns / fin	(respuesta-tiempo ?v)
Proteína	Preferencia	alta / media / baja / ns / fin	(respuesta-proteina ?v)
Densidad calórica	Preferencia	alta / media / baja / ns / fin	(respuesta-densidad-calorica ?v)

Cada respuesta se normaliza a minúsculas (`normalizar-respuesta`) y se valida contra su dominio antes de usarse.

4.2 Variables de salida del problema y su representación

La salida principal es la **receta recomendada**, junto con su justificación. Se representa con las siguientes plantillas y hechos:

- **RecetaCandidata** (`nombre ?n`) (`cf ?cf`): cada receta que sobrevive al filtrado, con su factor de certeza combinado. La receta de mayor `cf` es la recomendada.
- **RecetaOfertada** (`nombre ?n`): la receta concreta que se está proponiendo al usuario en ese momento.
- **RecetaRechazada** (`nombre ?n`) (`motivo ?m`): recetas que el usuario ha rechazado o descartado al reajustar un criterio.
- **evidencia** (`receta ?r`) (`factor ?f`) (`cf ?cf`) (`descripcion ?txt`): cada aporte de evidencia (a favor o en contra) sobre una receta; es la base de la explicación que se muestra.

La salida visible para el usuario es el texto impreso por `mostrar-receta` y `explicar-receta`: nombre, tipo, dificultad, comensales, tiempo, las restricciones que cumple, la evidencia a favor y en contra, las preguntas omitidas, la etiqueta de confianza (**MUY recomendable**, **recomendable**, ...) con su CF, y la probabilidad de acierto estimada.

4.3 Conocimiento global del sistema (carga inicial)

Al arrancar se cargan inicialmente:

- **El recetario** (BDrecetas_100.clp): hechos *receta* (nombre, tipo-plato, dificultad, comensales, tiempo-cocinado, info-nutricional, enlace-web) e *ingrediente* (nombre-receta, nombre-ingrediente, cantidad, unidad).
- **Los hechos de arranque** (defacts inicio-sistema-experto): el hecho (arrancar) y los nueve hechos (*pregunta ...*) que marcan qué información hay que pedir.
- **El conocimiento procedimental**: funciones auxiliares de clasificación de ingredientes, de extracción de datos nutricionales (*obtener-kcal*, *obtener-proteinas*), de combinación de factores de certeza (*combinar-cf*, fórmula de MYCIN), de lógica difusa (*membership*, *grado-densidad*) y de redacción de la explicación.
- **Las propiedades deducidas** (*propiedad_receta ...*): no se cargan, sino que se **generan** nada más ejecutar (*run*) por las reglas del Módulo 1, y a partir de ahí forman parte del conocimiento global usado por el resto de módulos.

4.4 Especificación de los módulos

4.4.1 Estructura en módulos

El sistema se organiza conceptualmente en cinco módulos, encadenados mediante un hecho de control (*fase ?*) que avanza por los valores *preguntar*, *elegir*, *puntuar*, *resultados*:

1. **Módulo 1 — Deducción de propiedades de las recetas** (base de conocimiento).
2. **Módulo 2 — Adquisición de información del usuario** (diálogo).
3. **Módulo 3 — Filtrado por restricciones esenciales** (conocimiento cierto).
4. **Módulo 4 — Valoración por factores de certeza** (incertidumbre).
5. **Módulo 5 — Propuesta de receta y reajustes**.

4.4.2 Módulo 1 — Deducción de propiedades de las recetas

- **Objetivo**: enriquecer cada receta con las propiedades necesarias para recomendar, deduciéndolas automáticamente antes del diálogo.
- **Conocimiento que utiliza**: los hechos *receta* e *ingrediente*, y las funciones de clasificación de ingredientes (*es-importante*, *es-dulce*, *contiene*, *obtener-kcal*, *obtener-proteinas*).
- **Conocimiento que deduce**: tipo de plato (si falta), ingrediente relevante, y las propiedades de , gluten, lactosa, no-vegetariana/no-vegana, vegetariana/vegana, sin gluten/sin lactosa, nivel de calorías y nivel de proteínas.
- **Hechos que utiliza**: (*receta ...*), (*ingrediente ...*).
- **Hechos que deduce**: (*propiedad_receta ingrediente_relevante ?r ?a*), (*propiedad_receta es_picante ?r*), (*propiedad_receta contiene_gluten ?r*), (*propiedad_receta contiene_lactosa ?r*), (*propiedad_receta no_vegetariana ?r*), (*propiedad_receta no_vegana ?r*), (*propiedad_receta es_vegetariana ?r*), (*propiedad_receta es_vegana ?r*), (*propiedad_receta es_sin_gluten ?r*),

(propiedad_receta es_sin_lactosa ?r), (propiedad_receta calorías baja|media|alta ?r), (propiedad_receta proteínas baja|media|alta ?r).

- **Reglas:** ingrediente-relevante-por-nombre, ingrediente-relevante-por-tipo, deducir-tipo-plato-postre, deducir-tipo-plato-principal, deducir-tipo-plato-por-defecto, marcar-picante, marcar-contiene-gluten, marcar-contiene-lactosa, marcar-no-vegetariana, marcar-no-vegana-por-otros, marcar-no-vegana-por-animal, marcar-no-vegana-por-lactosa, marcar-es-vegetariana, marcar-es-vegana, marcar-es-sin-gluten, marcar-es-sin-lactosa, clasificar-calorías, clasificar-proteínas.

4.4.3 Módulo 2 — Adquisición de información del usuario

- **Objetivo:** recoger del usuario las restricciones y preferencias, justificando cada pregunta y omitiendo las que no discriminan.
- **Conocimiento que utiliza:** los hechos (pregunta) pendientes, las funciones de validación de dominio y unico-campo-base (comprueba si todas las recetas compatibles comparten el mismo valor en un campo), para poder descartar preguntas no necesarias.
- **Conocimiento que deduce :** el perfil de necesidades del usuario (sus respuestas) y la lista de preguntas omitidas por no aportar información.
- **Hechos que utiliza:** (fase preguntar), (pregunta ?), (respuesta-dieta ?), etc.
- **Hechos que deduce:** los (respuesta-****) de la tabla de entradas, (pregunta-omitida (que ?)), (defaults-listos) y el avance de (fase ...).
- **Reglas:** mensaje-bienvenida, pregunta-dieta, pregunta-tipo, pregunta-sin-lactosa, pregunta-sin-gluten, pregunta-ingrediente-obligatorio, pregunta-comensales, pregunta-tiempo, pregunta-proteína, pregunta-densidad-calórica; sus respectivas validar-*; las reglas de omisión inteligente omitir-sin-lactosa-si-vegana, omitir-tipo-si-no-aporta, omitir-proteína-si-no-aporta, omitir-densidad-si-no-aporta, omitir-comensales-si-no-aporta, omitir-tiempo-si-no-aporta; y el control poner-default-ingrediente-pronto, finalizar-preguntas, finalizar-preguntas-por-fin, defaults-respuestas.

4.4.4 Módulo 3 — Filtrado por restricciones esenciales

- **Objetivo:** descartar las recetas que incumplen una restricción innegociable del usuario.
- **Conocimiento que utiliza:** las respuestas de tipo restricción y las propiedades deducidas en el Módulo 1.
- **Conocimiento que deduce :** el conjunto de recetas **candidatas** que cumplen todas las restricciones.
- **Hechos que utiliza:** (respuesta-dieta ?), (respuesta-sin-lactosa ?), (respuesta-sin-gluten ?), (respuesta-ingrediente-obligatorio ?), (propiedad_receta ...), (ingrediente ...).
- **Hechos que deduce y modifica:** crea las (RecetaCandidata ...) iniciales y **retira** las que incumplen una restricción; avanza la (fase ...).
- **Reglas:** iniciar-evaluación, filtrar-vegana, filtrar-vegetariana, filtrar-sin-lactosa, filtrar-sin-gluten, filtrar-ingrediente-obligatorio, pasar-a-puntuar.

4.4.5 Módulo 4 — Valoración por factores de certeza

- **Objetivo:** puntuar cada receta superviviente según las **preferencias** del usuario, generando evidencia (con CF) y combinándola con la fórmula de MYCIN.

- **Conocimiento que utiliza:** las respuestas de tipo preferencia, los datos de cada receta y las propiedades deducidas; funciones `cf-nivel`, `grado-densidad/membership` (lógica difusa), `combinar-cf` (MYCIN) y `acotar-cf`.
- **Conocimiento que deduce:** la evidencia a favor o en contra de cada receta y su factor de certeza combinado, si el usuario no expreso ninguna preferencia, se aplica el supuesto por defecto de preferir recetas fáciles.
- **Hechos que utiliza:** (`RecetaCandidata ...`), (`receta ...`), (`propiedad_receta ...`), (`respuesta-* ...`).
- **Hechos que deduce:** (`evidencia (receta ?) (factor ?) (cf ?) (descripcion ?)`), (`cf-calculada (receta ?)`), y la actualización del slot `cf` de cada `RecetaCandidata`.
- **Reglas:** `evidencia-por-defecto-dificultad` (razonamiento por defecto), `evidencia-tipo`, `evidencia-proteina`, `evidencia-calorias` (lógica difusa), `evidencia-comensales`, `evidencia-tiempo`, `combinar-cf-receta`, `pasar-a-resultados`.

4.4.6 Módulo 5 — Propuesta de receta y reajustes

- **Objetivo:** recomendar la receta de mayor CF, explicarla, y gestionar la interacción final (aceptar / rechazar / reajustar).
- **Conocimiento que utiliza:** las `RecetaCandidata` con su CF, las evidencias acumuladas, las preguntas omitidas y las restricciones del usuario y funciones `explicar-receta`, `etiqueta-confianza`, `prob-acierto-receta` (razonamiento probabilístico) y `unir-frases`.
- **Conocimiento que deduce:** la receta recomendada y su justificación en lenguaje natural si se rechaza, la siguiente mejor se selecciona, en caso de reajuste, relanza el razonamiento.
- **Hechos que utiliza:** (`fase resultados`), (`RecetaCandidata ...`), (`RecetaRechazada ...`), (`evidencia ...`), (`pregunta-omitida ...`), (`respuesta-final ?`).
- **Hechos que deduce:** (`RecetaOfertada ?`), (`RecetaRechazada ?`), (`respuesta-final ?`), (`fin`), y la reposición de (`fase elegir`) al reajustar.
- **Reglas:** `mostrar-receta`, `aceptar-receta`, `rechazar-receta`, `reajustar-criterio`, `sin-recetas-compatibles`, `sin-mas-recetas`, `cerrar-sistema`.

5 Manual de uso del sistema

1. **Carga del sistema.** Cargar primero el motor y después el recetario:

```
(load "SBC_recetas.clp")
(load "BDrecetas_100.clp")
```

El orden importa: si no se carga el recetario, el sistema avisa de que no hay recetas.

2. **Inicialización y ejecución.**

```
(reset)
(run)
```

(`reset`) carga los hechos iniciales y (`run`) lanza el razonamiento: primero se deducen las propiedades de las recetas y después comienza el diálogo.

3. **Responder a las preguntas.** El sistema irá preguntando una a una. Reglas de respuesta:
 - Se puede responder en mayúsculas o minúsculas.
 - `ns` = sin preferencia (esa pregunta no descarta ni puntúa).
 - `fin` = terminar de responder y recomendar ya con lo contestado hasta ese momento.
 - Antes de cada pregunta se explica por qué se hace. Algunas preguntas se omiten automáticamente si no aportan información.
4. **Interpretar la recomendación.** El sistema muestra la receta de mayor confianza con: tipo, dificultad, comensales, tiempo, las restricciones que cumple, la evidencia a favor y en contra, la etiqueta de confianza con su CF, y la probabilidad de acierto estimada.
5. **Decisión final.** Ante la receta propuesta se puede responder:
 - `a` (aceptar): se confirma la receta y termina.
 - `rechazar`: se descarta y se ofrece la siguiente mejor.
 - `dieta | tipo | comensales | tiempo | proteina | calorías | ingrediente`: reajustar ese criterio y relanzar el razonamiento.
6. **Caso sin recetas compatibles.** Si las restricciones dejan el recetario vacío, el sistema lo indica y ofrece modificar un parámetro o salir.